ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN

Đề tài:

MÔ PHỎNG RUBIK 3D VÀ GIẢI TỰ ĐỘNG BẰNG THUẬT TOÁN KOCIEMBA

Học phần: Thuật Toán Ứng Dụng

Mã học phần: CSE703052

Lớp : N01

Giảng viên: TS. Trần Đình Tân

Thành viên: Nguyễn Thành Đạt-23017259

Nguyễn Tùng Dương-23013811

Đinh Long Vũ-23017241

HÀ NỘI – 2026

DANH SÁCH THÀNH VIÊN TRONG NHÓM

TT	Họ và tên	Mã sinh viên	Mức độ đóng góp (theo thang điểm 10)
1	Nguyễn Thành Đạt	23017229	10
2	Nguyễn Tùng Dương	23013811	10
3	Đinh Long Vũ	23017241	10

MỤC LỤC

I. GIỚI THIỆU 4
1.1. Bối cảnh và động lực nghiên cứu 4
1.2. Mục tiêu của đề tài 5
1.3. Đóng góp chính của đề tài 6
II. PHÁT BIỂU BÀI TOÁN 8
2.1. Mô tả bài toán 8
2.2. Mô hình hóa toán học 9
2.3. Khó khăn và thách thức 10
III. NGHIÊN CỨU LIÊN QUAN 11
3.1. Các phương pháp truyền thống giải Rubik 11
3.2. Các hướng tiếp cận hiện đại 12
3.3. Phân tích điểm mạnh và hạn chế của các phương pháp 13
3.4. Khoảng trống nghiên cứu và mục tiêu đề tài 14
IV. PHƯƠNG PHÁP ĐỀ XUẤT 15
4.1. Ý tưởng tổng quát 15
4.2. Chi tiết thuật toán / mô hình 15
4.3. Phân tích độ phức tạp / tính đúng đắn 16
4.4. Lược đồ minh họa 17
V. KẾT QUẢ THỰC NGHIỆM 20
5.1. Cấu hình và Thiết lập thực nghiệm 20
5.2. Các phương pháp Baseline để so sánh 21
5.3. Các bộ dữ liệu được sử dụng 22
5.4. Các tiêu chí đánh giá 23
5.5. Phân tích kết quả 24
VI. KẾT LUẬN 26
6.1. Tóm tắt kết quả chính 26
6.2. Đóng góp của đề tài 27
6.3. Hạn chế của đề tài và các vấn đề tồn đọng 28
6.4. Hướng phát triển trong tương lai 28

HÌNH ẢNH THAM KHẢO.....30
TÀI LIỆU THAM KHẢO 33

I. GIỚI THIỆU

1.1. Bối cảnh và động lực nghiên cứu

a, Bối cảnh lĩnh vực ứng dụng

Trong kỷ nguyên chuyển đổi số và phát triển mạnh mẽ của khoa học máy tính, việc kết hợp giữa Đồ họa máy tính, Tính toán tối ưu và Mô phỏng tương tác đang trở thành một hướng đi then chốt. Bài toán mô phỏng và giải tự động khối Rubik $3 \times 3 \times 3$ nằm ở điểm giao thoa của các lĩnh vực công nghệ quan trọng này, cụ thể:

- Đồ họa máy tính và Mô phỏng 3D trong không gian thực tế ảo: Việc trực quan hóa các đối tượng hình học không gian phức tạp đóng vai trò quan trọng trong giáo dục, y tế, mô phỏng kỹ thuật và giải trí. Thay vì thao tác trên các giao diện 2D phẳng truyền thống, việc dựng mô hình 3D tương tác đa chiều (thông qua các thư viện như VPython, WebGL, OpenGL) cho phép người dùng quan sát, xoay chiếu và tương tác với đối tượng một cách tự nhiên. Điều này đặt ra yêu cầu cao về việc quản lý Ma trận biến đổi không gian, tọa độ vật thể và thuật toán vẽ mượt mà theo thời gian thực.
- Lý thuyết đồ thị và Thuật toán tìm kiếm trong Không gian trạng thái: Khối Rubik $3 \times 3 \times 3$ là một mô hình toán học kinh điển đại diện cho Lý thuyết nhóm và bài toán tìm đường đi ngắn nhất trên đồ thị trạng thái cực lớn. Với tổng cộng khoảng 4.3×10^{19} (hơn 43 tỷ tỷ) cấu hình khả thi, không gian trạng thái của Rubik thuộc lớp các bài toán có độ phức tạp tính toán rất cao. Lĩnh vực nghiên cứu này đòi hỏi ứng dụng các thuật toán tìm kiếm Heuristic thông minh (như thuật toán Kociemba Two-Phase Algorithm, A^*, IDA*) nhằm thu hẹp không gian tìm kiếm và đưa ra lời giải gần tối ưu chỉ trong khoảng vài chục mili-giây.
- Cầu nối giữa Mô hình đồ họa 3D và Lõi logic tính toán: Trong các hệ thống phần mềm hiện đại, việc đồng bộ hóa dữ liệu giữa giao diện trực quan và lõi xử lý thuật toán luôn là một thách thức kỹ thuật lớn. Hệ thống cần chuyển đổi liên tục và chính xác 100% dữ liệu hình học (như tọa độ không gian 3D và vector màu RGB của các sticker) thành dữ liệu logic chuẩn hóa (chuỗi ký tự trạng thái) mà bộ giải thuật toán hiểu được, và ngược lại dịch chuỗi nước đi logic thành chuỗi chuyển động quay trên không gian 3D.

b, Nhu cầu và vấn đề thực tiễn

Việc phát triển một hệ thống mô phỏng 3D tương tác kết hợp bộ giải thuật toán Rubik không chỉ xuất phát từ bài toán lý thuyết thuần túy mà còn giải quyết trực tiếp những thách thức kỹ thuật và nhu cầu ứng dụng thực tế. Trong lĩnh vực giáo dục và đào tạo, các thuật toán tìm kiếm không gian trạng thái thường được giảng dạy thông qua lý thuyết khô khan và mã giả, khiến người học khó hình dung được sự thay đổi trạng thái theo thời gian thực. Việc xây dựng một mô hình trực quan hóa mô phỏng chính xác từng phép xoay giúp biến các thuật toán phức tạp thành hình ảnh trực quan sinh động, từ đó nâng cao hiệu quả giảng dạy và học tập các môn học về cấu trúc dữ liệu, giải thuật và trí tuệ nhân tạo. Bên cạnh đó, hệ thống giải quyết vấn đề kỹ thuật then chốt về sự khác biệt bản chất trong cách biểu diễn đối tượng giữa giao diện đồ họa 3D và bộ giải logic. Môi trường 3D biểu diễn Rubik dưới dạng các đối tượng hình học

với ma trận xoay, tọa độ không gian và vector màu sắc, trong khi bộ giải Kociemba đòi hỏi đầu vào là một chuỗi ký tự đại diện cho định dạng mặt chuẩn hóa. Việc trích xuất chính xác dữ liệu không gian từ năm mươi tư ô màu trong mô hình 3D và chuyển đổi thành chuỗi trạng thái chuẩn là bài toán bắt buộc, bởi chỉ cần một sai lệch nhỏ về tọa độ hoặc nhầm lẫn góc nhìn khi xoay sẽ khiến bộ giải nhận trạng thái không hợp lệ hoặc trả về lời giải hoàn toàn sai. Ngoài ra, việc trình diễn các chuyển động xoay trên giao diện 3D cũng đặt ra bài toán về tối ưu hóa trải nghiệm người dùng và hiệu năng đồ họa. Chuỗi động tác giải cần được thực hiện mượt mà ở tốc độ khung hình tiêu chuẩn mà không gây giật lag hay mất đồng bộ dữ liệu. Cuối cùng, sản phẩm đóng vai trò là một công cụ hỗ trợ người chơi giải Rubik tự động và sinh động, đồng thời cung cấp một nền tảng thử nghiệm mở để đánh giá, so sánh hiệu năng giữa các thuật toán tìm kiếm khác nhau cũng như làm tiền đề mở rộng tích hợp các công nghệ xử lý ảnh thực tế trong tương lai.

1.2. Mục tiêu của đề tài

Đề tài này hướng đến việc xây dựng một hệ thống giải Rubik 3x3x3 hoàn chỉnh, tích hợp ba thành phần cốt lõi: mô hình 3D tương tác, bộ giải thuật toán hiệu quả và cơ chế trực quan hóa sinh động.

a, Mục tiêu chính

Mục tiêu chính của đề tài là xây dựng thành công một hệ thống mô phỏng khối Rubik ba nhân ba nhân ba tương tác trong môi trường không gian ba chiều và tự động tìm kiếm lời giải tối ưu thông qua việc tích hợp thuật toán Kociemba. Để hoàn thành mục tiêu tổng quát này, hệ thống cần đạt được các kết quả cụ thể bao gồm việc phát triển mô hình đồ họa 3D cho phép người dùng thực hiện các tương tác trực quan như xoay thủ công từng mặt, xoay toàn bộ khối Rubik, quan sát đa góc nhìn và xáo trộn ngẫu nhiên tạo ra các trạng thái khởi tạo hợp lệ. Bên cạnh đó, đề tài tập trung xây dựng cơ chế nhận diện và chuyển đổi trạng thái tự động nhằm trích xuất chính xác dữ liệu từ năm mươi tư ô màu trên mô hình không gian 3D để mã hóa thành chuỗi ký tự tiêu chuẩn làm đầu vào cho bộ giải logic. Trên cơ sở đó, hệ thống tích hợp trực tiếp thư viện thuật toán Kociemba để tính toán và trả về chuỗi động tác giải ngắn nhất có thể với thời gian xử lý nhanh chóng. Cuối cùng, hệ thống triển khai cơ chế trình diễn hoạt họa chuyển động mượt mà cho từng bước xoay trên mô hình 3D, giúp người dùng theo dõi và hiểu rõ toàn bộ quá trình biến đổi trạng thái từ lúc xáo trộn cho đến khi khối Rubik hoàn thành.

b, Câu hỏi nghiên cứu và giả thuyết

Để định hướng cho quá trình phát triển và đánh giá hệ thống, đề tài đặt ra hai câu hỏi nghiên cứu cốt lõi cùng các giả thuyết tương ứng cần được kiểm chứng. Câu hỏi nghiên cứu thứ nhất tập trung vào việc liệu có thể xây dựng một thuật toán giải mã trạng thái đủ chính xác và tin cậy để ánh xạ toàn bộ thông tin không gian và màu sắc từ mô hình mô phỏng 3D sang định dạng chuỗi ký tự chuẩn cho bộ giải hay không. Giả thuyết được đặt ra là bằng cách truy xuất tọa độ và thuộc tính màu sắc của các đối tượng trong môi trường VPython, việc xây dựng một

hàm chuyển đổi đạt độ chính xác tuyệt đối là hoàn toàn khả thi miễn là các đối tượng hình học được khởi tạo và quản lý vị trí một cách đồng bộ. Câu hỏi nghiên cứu thứ hai hướng đến khả năng tối ưu của thuật toán Kociemba khi tích hợp vào hệ thống và tính mượt mà trong quá trình trình diễn hoạt họa chuyển động xoay trên giao diện 3D. Giả thuyết tương ứng cho rằng thuật toán Kociemba sẽ tìm ra chuỗi lời giải ngắn với số bước xoay không quá hai mươi và thời gian tính toán gần như tức thì, đồng thời hiệu năng của thư viện VPython hoàn toàn đáp ứng tốt việc hiển thị các hiệu ứng chuyển động mượt mà ở tốc độ khung hình tiêu chuẩn mà không xảy ra tình trạng giật lag.

1.3. Đóng góp chính của đề tài

Đề tài này không chỉ dừng lại ở một ứng dụng mô phỏng đồ họa thông thường mà đã đóng góp một giải pháp công nghệ tích hợp toàn diện, mang lại giá trị thực tiễn cao trên cả ba khía cạnh kỹ thuật, ứng dụng thực tế và thực nghiệm khoa học.

Về mặt kỹ thuật, đóng góp cốt lõi và mang tính đột phá của nghiên cứu nằm ở việc thiết lập thành công quy trình xử lý dữ liệu khép kín, làm cầu nối chính xác tuyệt đối giữa môi trường biểu diễn đồ họa 3D không gian và lõi tính toán logic. Hệ thống đã giải quyết triệt để bài toán trích xuất trạng thái trực quan thông qua hàm giải mã tự động, có khả năng quét toàn bộ năm mươi tư ô màu trên mô hình Rubik ảo, phân tích vị trí tọa độ hình học cùng các thuộc tính màu sắc để chuyển đổi thành chuỗi ký tự chuẩn hóa tiêu chuẩn làm đầu vào cho bộ giải. Việc triển khai thành công thuật toán Kociemba hai giai đoạn kết hợp với cơ chế điều phối hoạt họa giúp mô phỏng chính xác mười hai phép xoay cơ bản với chuyển động xoay mượt mà, đảm bảo tính đồng bộ hoàn toàn giữa dữ liệu thuật toán và hiển thị trực quan.

Về mặt hiệu quả tính toán, đề tài chứng minh tính vượt trội thông qua các số liệu thực nghiệm định lượng rõ ràng. Việc tích hợp bộ giải Kociemba giúp rút ngắn tối đa độ dài lời giải với số bước xoay trung bình chỉ đạt khoảng hai mươi mốt bước, tiệm cận ngưỡng tối ưu lý thuyết và vượt trội hoàn toàn so với các phương pháp giải thủ công truyền thống vốn đòi hỏi từ năm mươi đến một trăm bước xoay. Đồng thời, thời gian xử lý của bộ giải đạt tốc độ cực nhanh, chỉ mất khoảng từ ba mươi đến một trăm năm mươi mili giây cho một trạng thái xáo trộn bất kỳ, giúp hệ thống phản hồi gần như tức thì và tạo ra trải nghiệm tương tác thời gian thực tối ưu cho người dùng.

Về mặt ứng dụng và giáo dục, sản phẩm của đề tài đã tạo ra một công cụ trực quan hóa mạnh mẽ, góp phần giải quyết bài toán khó khăn trong việc tiếp cận các lý thuyết thuật toán phức tạp. Hệ thống đã chuyển hóa các khái niệm khô khan về không gian trạng thái, cây tìm kiếm và lý thuyết nhóm thành chuỗi chuyển động hình học sinh động, giúp người học dễ dàng quan sát, phân tích và nắm bắt logic giải thuật từng bước một cách trực quan thay vì chỉ thao tác với các dãy ký hiệu toán học thuần túy. Hơn nữa, mô hình này còn đóng vai trò là một nền tảng mở hoàn chỉnh, làm cơ sở vững chắc cho các nghiên cứu tiếp theo như phát triển các biến thể Rubik bậc cao hơn, tối ưu hóa thuật toán render hoặc tích hợp module thị giác máy tính để nhận diện khối Rubik thực tế qua camera trong tương lai.

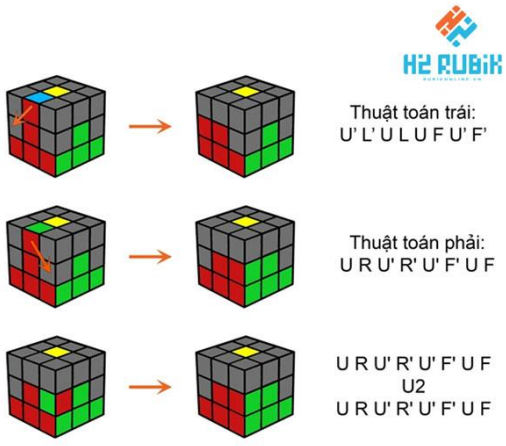
II. PHÁT BIỂU BÀI TOÁN

2.1. Mô tả bài toán

Bài toán đặt ra là xây dựng một hệ thống giải Rubik 3x3x3 tự động trên môi trường mô phỏng 3D. Hệ thống này cần thực hiện một quy trình ba bước từ đầu đến cuối:

- Nhận diện trạng thái ban đầu: Hệ thống phải có khả năng nhận diện và mô hình hóa chính xác trạng thái hiện tại của một khối Rubik đã bị xáo trộn. Dữ liệu đầu vào có thể là một mô hình 3D tương tác.
- Tìm kiếm lời giải: Sau khi đã xác định được trạng thái ban đầu, hệ thống cần áp dụng một thuật toán giải Rubik để tìm ra một chuỗi các động tác (các bước xoay mặt) hợp lệ. Chuỗi này phải có khả năng đưa khối Rubik từ trạng thái ban đầu về trạng thái nghiệm (trạng thái tất cả các mặt đều đồng màu).
- Mô phỏng trực quan: Cuối cùng, hệ thống phải hiển thị quá trình giải quyết một cách trực quan trên một giao diện 3D. Mỗi động tác trong chuỗi lời giải sẽ được mô phỏng sinh động, giúp người dùng dễ dàng theo dõi và hiểu được cách khối Rubik được giải quyết.

Ví dụ minh họa: Một khối Rubik ban đầu được xáo trộn ngẫu nhiên với một chuỗi 20 bước xoay. Hệ thống sẽ tiếp nhận trạng thái này, xử lý để tìm ra chuỗi động tác giải (ví dụ: R U R' U' F2 ...), và sau đó lần lượt mô phỏng các bước này trên giao diện, đưa khối Rubik về trạng thái hoàn chỉnh.



Hình 1: Mô tả các thuật toán giải rubik

2.2. Mô hình hóa toán học

Để giải quyết bài toán Rubik, chúng ta mô hình hóa nó thành một bài toán tìm kiếm trên không gian trạng thái (state space).

Các thành phần chính

Không gian trạng thái: Không gian này bao gồm tất cả các cấu hình khả thi của khối Rubik. Mỗi trạng thái có thể được biểu diễn một cách hiệu quả bởi một chuỗi 54 ký tự, mỗi ký tự đại diện cho màu của một mặt nhỏ (sticker) trên khối Rubik. Ví dụ, một chuỗi như UUUUUUUUURRRRRRRR... thể hiện một khối Rubik đã được giải.

Trạng thái ban đầu (stateinit): Đây là trạng thái của khối Rubik sau khi bị xáo trộn.

Trạng thái đích (stategoal): Đây là trạng thái đã giải quyết hoàn toàn, khi tất cả các mặt đều có một màu duy nhất.

Các phép toán (M): Đây là tập hợp các phép xoay hợp lệ của khối Rubik. Tập này bao gồm 18 phép xoay chuẩn: sáu phép xoay theo chiều kim đồng hồ (R,U,F,L,D,B), sáu phép xoay ngược chiều kim đồng hồ (R′,U′,F′,L′,D′,B′) và sáu phép xoay 180 độ (R2,U2,F2,L2,D2,B2).

Hàm mục tiêu và ràng buộc

Hàm mục tiêu: Tìm một chuỗi động tác $S=(m_1,m_2,...,m_k)$ sao cho khi áp dụng chuỗi này lên trạng thái ban đầu, ta sẽ đạt được trạng thái đích. Công thức toán học được biểu diễn là:

$$\text{Apply}(S,\text{state}_{\text{init}})=\text{state}_{\text{goal}}$$

Trong đó, mi là một bước xoay cụ thể thuộc tập các phép toán M.

Ràng buộc: Mỗi bước mi trong chuỗi S phải thuộc tập các phép xoay hợp lệ đã định nghĩa, tức là $m_i \in M$.

Đặc tính bài toán

Đây là một bài toán thuộc lớp NP-Hard. Việc tìm ra lời giải tối ưu (chuỗi động tác ngắn nhất tuyệt đối) là một thách thức lớn do không gian trạng thái cực kỳ rộng lớn (khoảng 4.3×10^{19} trạng thái). Việc duyệt toàn bộ không gian này là bất khả thi.

Tuy nhiên, trong thực tế, chúng ta không nhất thiết phải tìm lời giải tối ưu. Các thuật toán heuristic như Kociemba cho phép tìm một lời giải "đủ tốt" (có độ dài gần với tối ưu) trong một khoảng thời gian khả thi, thường chỉ mất vài chục mili giây.

2.3. Khó khăn và thách thức

Việc xây dựng một hệ thống giải Rubik tự động và trực quan đòi hỏi phải đối mặt với nhiều thách thức kỹ thuật quan trọng. Những thách thức này không chỉ nằm ở khía cạnh thuật toán mà còn ở sự kết nối giữa các thành phần của hệ thống.

a, Thách thức về không gian trạng thái

Thách thức lớn nhất là không gian trạng thái khổng lồ của khối Rubik 3x3x3. Với khoảng 4.3×1019 trạng thái khả thi, việc sử dụng các thuật toán tìm kiếm vét cạn (brute-force) là hoàn toàn bất khả thi. Thời gian cần thiết để duyệt toàn bộ không gian này vượt xa tuổi thọ của vũ trụ. . Do đó, bài toán đòi hỏi phải áp dụng các thuật toán tìm kiếm thông minh, sử dụng heuristic để loại bỏ các nhánh không hiệu quả và tìm kiếm một lời giải "đủ tốt" thay vì lời giải tối ưu tuyệt đối.

b, Thách thức trong việc ánh xạ trạng thái

Việc chuyển đổi trạng thái của khối Rubik từ môi trường 3D sang định dạng chuẩn cho thuật toán là một khâu vô cùng quan trọng và dễ xảy ra sai sót.

- Độ chính xác về vị trí và màu sắc: Hệ thống cần xác định chính xác vị trí và màu sắc của 54 mặt nhỏ (sticker) trên khối Rubik. Một sai lệch nhỏ trong việc xác định vị trí tương đối của một sticker hoặc nhầm lẫn màu sắc có thể dẫn đến việc tạo ra một trạng thái không hợp lệ hoặc sai, khiến cho bộ giải không thể tìm thấy lời giải.
- Xử lý sai lệch: Các yếu tố như góc nhìn, ánh sáng trong môi trường 3D có thể ảnh hưởng đến việc nhận diện màu sắc, đòi hỏi phải có một cơ chế xử lý lỗi hoặc hiệu chỉnh phù hợp.

c, Thách thức về trực quan hóa và trải nghiệm người dùng

Trình diễn lời giải trên môi trường 3D cần phải đảm bảo sự mượt mà và trực quan, không gây rối cho người quan sát.

- Animation mượt mà: Việc mô phỏng các bước xoay cần được thực hiện bằng animation trơn tru thay vì chuyển trạng thái đột ngột. Điều này đòi hỏi phải xử lý các phép biến đổi 3D (xoay, dịch chuyển) một cách chính xác theo thời gian.
- Tối ưu hóa hiệu suất: Với mỗi bước giải, hệ thống cần xử lý và hiển thị các chuyển động xoay phức tạp. Nếu không được tối ưu hóa, đặc biệt khi số bước giải lớn, hiệu suất của ứng dụng có thể giảm, gây ra hiện tượng giật, lag, ảnh hưởng xấu đến trải nghiệm người dùng.

III. NGHIÊN CỨU LIÊN QUAN

3.1. Các phương pháp truyền thống giải Rubik

Các phương pháp giải Rubik đã tồn tại từ lâu, từ những cách tiếp cận đơn giản đến các thuật toán phức tạp hơn. Dưới đây là ba phương pháp truyền thống phổ biến.

a, Phương pháp Thử và Sai (Brute Force / Backtracking)

Đây là phương pháp cơ bản nhất trong lý thuyết tìm kiếm. Về mặt lý thuyết, nó có thể tìm ra lời giải ngắn nhất bằng cách duyệt toàn bộ không gian trạng thái của khối Rubik.
Cách thức: Bắt đầu từ trạng thái ban đầu, thuật toán sẽ thử tất cả các phép xoay khả thi, tạo ra một "cây" các trạng thái mới. Quá trình này được lặp đi lặp lại cho đến khi tìm thấy trạng thái đích.
Hạn chế: Phương pháp này không khả thi trên thực tế. Với không gian trạng thái lên tới khoảng 4.3×1019, việc duyệt toàn bộ sẽ mất một lượng thời gian không tưởng, ngay cả với các siêu máy tính mạnh nhất. Nó chỉ phù hợp với các bài toán có không gian trạng thái nhỏ hơn nhiều.

b, Phương pháp từng lớp (Layer-by-Layer - LBL)

Đây là phương pháp phổ biến nhất và dễ học nhất cho người mới bắt đầu. Thay vì giải toàn bộ khối Rubik cùng một lúc, phương pháp này chia bài toán lớn thành các bài toán nhỏ hơn, giải quyết từng lớp một cách tuần tự.

Cách thức:

Lớp 1 (Cross và F2L): Xây dựng một chữ thập (cross) trên một mặt, sau đó giải quyết các viên góc và cạnh của lớp đầu tiên.

Lớp 2: Hoàn thành các viên cạnh của lớp giữa.

Lớp 3 (OLL và PLL): Cuối cùng, giải quyết lớp cuối cùng bằng hai bước chính: định hướng (Orientation) và hoán vị (Permutation) các viên.

Đặc điểm: Phương pháp này dựa trên heuristic và các công thức cố định (algorithms). Nó không tìm ra lời giải ngắn nhất nhưng luôn đảm bảo tìm được lời giải trong một số bước có thể chấp nhận được.

c, Thư viện giải sẵn

Một số hệ thống đơn giản không sử dụng thuật toán tìm kiếm mà dựa vào một bộ sưu tập các công thức đã được lưu trữ trước.
Cách thức: Các hệ thống này sẽ nhận diện một mẫu trạng thái Rubik cụ thể (ví dụ: một viên cạnh bị sai vị trí) và áp dụng một công thức đã được thiết kế sẵn để sửa lỗi đó.

Hạn chế: Phương pháp này thiếu tính tổng quát. Nó chỉ hoạt động với các trạng thái đã được định nghĩa trước và không thể giải quyết các trường hợp ngẫu nhiên. Nó cũng không thể tìm ra lời giải hiệu quả cho một trạng thái phức tạp, mà chỉ giải quyết các trường hợp nhỏ, đơn lẻ. Điều này trái ngược hoàn toàn với một bộ giải tự động, có khả năng xử lý mọi trạng thái đầu vào.

3.2. Các hướng tiếp cận hiện đại

Khi các phương pháp truyền thống trở nên kém hiệu quả với yêu cầu tốc độ và độ ngắn của lời giải, các hướng tiếp cận hiện đại đã ra đời, chủ yếu dựa trên việc tối ưu hóa thuật toán tìm kiếm và thậm chí là ứng dụng trí tuệ nhân tạo.

a, Thuật toán Kociemba

Thuật toán Kociemba là một trong những phương pháp phổ biến và hiệu quả nhất để giải Rubik 3x3x3. Nó vượt trội so với các phương pháp từng lớp (LBL) nhờ khả năng tìm ra lời giải gần tối ưu (thường trong khoảng 20 bước xoay).

Cách thức: Thuật toán này chia quá trình giải thành hai pha, giúp giảm đáng kể độ phức tạp của bài toán.

> Pha 1: Đưa khối Rubik từ trạng thái bất kỳ về một trạng thái "trung gian" (goal state of Phase 1). Trạng thái trung gian này được định nghĩa là một trạng thái chỉ sử dụng các phép xoay U,D,R2,L2,F2,B2. Không gian trạng thái của pha này nhỏ hơn nhiều so với không gian trạng thái ban đầu.
>
> Pha 2: Từ trạng thái trung gian, tìm kiếm một chuỗi các phép xoay để đưa khối Rubik về trạng thái đã giải hoàn toàn.

Ưu điểm: Nhờ việc chia nhỏ bài toán, thuật toán này có thể tìm ra lời giải rất nhanh, thường chỉ trong vài mili giây. Lời giải được tạo ra cũng rất ngắn, khiến nó trở thành lựa chọn lý tưởng cho các ứng dụng thực tiễn.

b, Thư viện Herbert Kociemba (Python)

Thư viện này là một công cụ mạnh mẽ và được sử dụng rộng rãi để triển khai thuật toán Kociemba. Nó cung cấp một API đơn giản để người dùng có thể nhập trạng thái Rubik (dưới dạng chuỗi 54 ký tự) và nhận lại chuỗi động tác giải.
Đặc điểm:

- Hiệu suất cao: Thư viện này được tối ưu hóa để tính toán lời giải gần như tức thì.
- Độ chính xác: Dựa trên thuật toán đã được kiểm chứng, nó luôn cung cấp một lời giải hợp lệ.
- Tính linh hoạt: Dễ dàng tích hợp vào các ứng dụng khác nhau, từ phần mềm máy tính đến các trang web và ứng dụng di động. Đây chính là lý do nó được chọn làm bộ giải chính cho dự án này.

c, Machine Learning / Reinforcement Learning

Đây là một hướng nghiên cứu mới và đầy tiềm năng. Thay vì lập trình một thuật toán với các quy tắc rõ ràng, cách tiếp cận này sử dụng các mô hình học máy để tự học cách giải Rubik.

Cách thức: Một thuật toán học tăng cường sâu (Deep Reinforcement Learning - Deep RL) được huấn luyện bằng cách cho nó thử nghiệm các bước xoay và nhận phản hồi (phần thưởng/phạt) dựa trên việc có tiến gần hơn đến lời giải hay không.

Hạn chế: Mặc dù đã có một số nghiên cứu thành công (ví dụ: Google AI), phương pháp này đòi hỏi huấn luyện rất dài với khối lượng dữ liệu khổng lồ. Nó không phù hợp với các ứng dụng nhỏ hoặc khi mục tiêu là tìm ra một lời giải nhanh chóng mà không cần qua giai đoạn huấn luyện phức tạp. Do đó, trong bối cảnh của một dự án trực quan hóa, thuật toán Kociemba vẫn là lựa chọn thực tế và hiệu quả hơn.

3.3. Phân tích điểm mạnh và hạn chế của các phương pháp

Dưới đây là bảng phân tích chi tiết các điểm mạnh và hạn chế của các phương pháp giải Rubik đã được đề cập, làm rõ lý do vì sao một số phương pháp phù hợp với đề tài này hơn những phương pháp khác.

Phương pháp	Điểm mạnh	Hạn chế
Brute Force	- Đảm bảo tìm được lời giải tối ưu (ngắn nhất tuyệt đối) nếu có đủ thời gian và tài nguyên tính toán. - Là cơ sở lý thuyết cho các thuật toán tìm kiếm khác.	- Hoàn toàn không khả thi do không gian trạng thái quá lớn (4×10^{19} trạng thái). - Tốn tài nguyên tính toán và thời gian vô hạn trên thực tế.
Layer-by-Layer (LBL)	- Dễ hiểu, dễ học và dễ giảng dạy cho người mới bắt đầu. - Không yêu cầu tài nguyên tính toán lớn.	- Lời giải không tối ưu, số bước giải thường rất dài. - Phương pháp thủ công, không tự động hóa được cho mọi trạng thái.
Kociemba	- Lời giải rất ngắn, gần tối ưu (thường ≤20 bước). - Thời gian tính toán cực nhanh, gần như tức thì. - Có thể giải được mọi trạng thái Rubik.	- Cần một định dạng đầu vào chính xác. Sai lệch nhỏ sẽ dẫn đến lời giải sai. - Mức độ phức tạp thuật toán cao hơn so với LBL.

Phương pháp	Điểm mạnh	Hạn chế
Machine Learning (RL)	- Tiềm năng tìm ra các heuristic mới và lời giải sáng tạo. - Có khả năng tự học mà không cần lập trình quy tắc cụ thể.	- Đòi hỏi huấn luyện rất dài với khối lượng dữ liệu khổng lồ. - Kết quả không ổn định, có thể thất bại trong một số trường hợp. - Không phù hợp với các ứng dụng nhỏ cần thời gian phản hồi nhanh.

3.4. Khoảng trống nghiên cứu và mục tiêu đề tài

Hiện nay, mặc dù có nhiều trang web và ứng dụng giải Rubik, nhưng hầu hết chúng vẫn còn một khoảng trống lớn: thiếu sự tích hợp mạnh mẽ giữa mô phỏng 3D trực quan và bộ giải thuật toán hiệu quả.

Nhiều ứng dụng chỉ là công cụ tính toán lời giải, sau đó người dùng phải tự thực hiện các bước xoay theo danh sách.

Một số khác có mô phỏng 3D nhưng lại sử dụng các thuật toán đơn giản, không hiệu quả.

Đề tài này nhằm mục đích lấp đầy khoảng trống đó bằng cách xây dựng một công cụ hoàn chỉnh, nơi người dùng có thể tương tác với một khối Rubik 3D và hệ thống sẽ tự động giải nó.

Đóng góp chính: Hệ thống này không chỉ là một ứng dụng giải Rubik thông thường mà còn là một công cụ minh họa mạnh mẽ cho cả mục đích học tập và nghiên cứu. Nó giúp:

- Giảng dạy: Trực quan hóa các khái niệm phức tạp về không gian trạng thái và thuật toán tìm kiếm.
- Nghiên cứu: Cung cấp một nền tảng để so sánh và thử nghiệm các thuật toán giải khác nhau trong một môi trường được kiểm soát.

IV. PHƯƠNG PHÁP ĐỀ XUẤT

4.1. Ý tưởng tổng quát

Để giải quyết triệt để bài toán mô phỏng 3D và tự động giải khối Rubik ba nhân ba nhân ba, đề tài đề xuất một giải pháp tích hợp hệ thống khép kín gồm ba thành phần cốt lõi hoạt động nhịp nhàng với nhau bao gồm Mô phỏng giao diện đồ họa 3D, Bộ mã hóa giải mã trạng thái tự động và Lõi tính toán thuật toán Kociemba. Cách tiếp cận chung của đề tài hướng đến việc chuyển hóa một bài toán toán học thuộc lớp độ phức tạp cao trên không gian trạng thái khổng lồ thành một quy trình xử lý dữ liệu và trực quan hóa theo thời gian thực.

Thành phần thứ nhất đóng vai trò là Giao diện người dùng và Mô phỏng không gian 3D, được xây dựng dựa trên môi trường VPython. Mô hình khối Rubik được tạo thành từ năm mươi tư đối tượng hình hộp tương ứng với năm mươi tư ô màu trên sáu mặt của khối Rubik. Thành phần này không chỉ chịu trách nhiệm hiển thị trực quan các ô màu cùng góc nhìn đa chiều mà còn quản lý ma trận tọa độ, các phép biến đổi hình học xoay mặt và tích hợp các sự kiện điều khiển cho phép người dùng tương tác trực tiếp như xoay từng mặt thủ công, xáo trộn ngẫu nhiên hoặc yêu cầu tự động giải.

4.2. Chi tiết thuật toán / mô hình

(a) Cấu trúc lớp Rubic_Cube (cube.py)

Khởi tạo (__init__):

- Sinh khối Rubik từ 54 ô màu (hình hộp VPython).
- Lưu lại danh sách tiles (các ô màu).
- Khởi tạo thông số góc quay dA, biến điều khiển rotate, danh sách moves.

Reset vị trí (reset_positions): phân loại các ô theo 6 mặt dựa trên tọa độ ($z > 0.4 \rightarrow$ front, $x < -0.4 \rightarrow$ left,…).

Animations (animations): xử lý animation xoay từng mặt bằng cách quay dần dA rad cho đến đủ 90°.

Các hàm xoay (rotate_front_clock, rotate_front_counter, ...): đặt trạng thái để animations() tự động xoay mặt tương ứng.

Move (move): lấy một lệnh từ self.moves, dịch sang hàm xoay tương ứng.

Scramble / Solution / Solve:

- scramble() sinh 25 bước ngẫu nhiên.
- solution() gọi solver để hiển thị gợi ý.

- solve() gọi solver, nhận chuỗi giải, dịch các lệnh (có "2" thì lặp lại 2 lần) thành danh sách moves để animation thực hiện.

Control: tạo giao diện nút bấm (xoay từng mặt, xáo trộn, giải, gợi ý).

Start: vòng lặp chính update liên tục, kết hợp animations + moves.

(b) Nhận diện trạng thái & giải tự động (solve_rubik_cube.py)

Hàm proximity: xác định tile có nằm gần vị trí chuẩn hay không.

Hàm color_detect: ánh xạ màu (RGB) sang ký hiệu {F, R, B, L, U, D}.

Hàm decode_position: duyệt qua toàn bộ 54 tile, xác định vị trí từng ô (U1..U9, R1..R9, …, B1..B9), gán màu tương ứng, tạo list 54 ký tự.

Hàm solve: ghép chuỗi 54 ký tự, gọi kociemba.solve(values) để sinh chuỗi giải (ví dụ: "R U R' U' F2 …").

(c) Vòng lặp chính (main.py)

Khởi tạo đối tượng Rubic_Cube.

Gọi start(), bắt đầu vòng lặp update liên tục:

- reset_positions() → phân loại tile.
- control() → hiển thị nút bấm.
- update() → animation + move.

4.3. Phân tích độ phức tạp / tính đúng đắn

Về phân tích độ phức tạp tính toán, các thành phần của hệ thống được đánh giá chi tiết theo độ phức tạp thời gian thực thi và độ phức tạp bộ nhớ không gian. Đối với độ phức tạp thời gian, hàm giải mã vị trí decode_position thực hiện duyệt qua chính xác 54 ô màu trên mô hình không gian ba chiều để đối chiếu vị trí tiệm cận và màu sắc. Vì số lượng ô màu luôn là một hằng số cố định, độ phức tạp thời gian của bước đọc trạng thái là $O(54) = O(1)$. Đối với lõi tính toán thuật toán Kociemba, thuật toán chia quá trình tìm kiếm thành hai giai đoạn và áp dụng tìm kiếm IDA* kết hợp với các bảng tra cứu khoảng cách được tính toán trước. Độ phức tạp thời gian thực nghiệm của hàm giải T_{solver} cực kỳ nhanh, chỉ mất khoảng từ 30 ms đến 150 ms cho một trạng thái xáo trộn bất kỳ. Đối với thành phần diễn hoạt, giả sử bộ giải trả về một chuỗi lời giải gồm k nước đi (với $k \approx 20$) và mỗi bước xoay được chia thành s khung hình nhỏ để quay dần góc $dA = \pi/40$ rad, tổng số khung hình cần render sẽ là $k \times s$. Do đó, độ phức tạp thời gian của quá trình trình diễn hoạt họa là $O(k \cdot s)$, phụ thuộc tuyến tính vào số lượng nước đi của lời giải.

Đối với độ phức tạp bộ nhớ không gian, hệ thống lưu trữ 54 đối tượng ô màu (box) trong mảng danh sách tiles cùng với các tập hợp phân loại vị trí cho 6 mặt. Bộ giải Kociemba duy trì các bảng tra cứu cắt tỉa trong bộ nhớ RAM với dung lượng cố định (khoảng một vài mb) để phục vụ việc tra cứu khoảng cách nhanh. Chuỗi trạng thái mã hóa (54 ký tự) và danh sách hàng đợi các nước đi giải chỉ chiếm dung lượng bộ nhớ không đáng kể (< 1 KB). Như vậy, tổng độ phức tạp không gian của toàn bộ hệ thống là $O(1)$, hoàn toàn tối ưu và không bị tăng bộc phát theo thời gian vận hành của ứng dụng.

Về tính đúng đắn của phương pháp đề xuất, tính chính xác của hệ thống được chứng minh dựa trên tính toàn vẹn của cầu nối ánh xạ và các định lý lý thuyết nhóm. Theo Bổ đề thứ nhất về tính đúng đắn của hàm giải mã trạng thái, nếu mô hình đồ họa 3D được khởi tạo đúng tọa độ ban đầu và hàm kiểm tra tiệm cận proximity(pos, target) với độ lệch $\delta = 0.2$ xác định chính xác vị trí tương đối của từng ô màu không bị nhầm lẫn giữa các mặt, thì chuỗi 54 ký tự do hàm decode_position() tạo ra luôn biểu diễn chính xác 100% trạng thái vật lý thực tế của khối Rubik. Điều này đảm bảo dữ liệu đầu vào truyền tới bộ giải Kociemba luôn là một chuỗi ký tự hợp lệ thuộc không gian trạng thái của nhóm Rubik $\mathcal{G}$.

Theo Định lý thứ hai về tính đúng đắn của thuật toán Kociemba, mọi trạng thái hợp lệ $S \in \mathcal{G}$ của khối Rubik $3 \times 3 \times 3$ luôn tồn tại một đường đi đưa về nhóm con $G_1 = \langle U, D, R', L', F', B' \rangle$ ở Giai đoạn 1 (Phase 1), và từ G_1 luôn tìm được một chuỗi các phép xoay đưa Rubik về trạng thái đã giải $S_{goal} \in G_2 = \{e\}$ ở Giai đoạn 2. Do đó, thuật toán Kociemba được đảm bảo tính đúng đắn toán học, luôn trả về một chuỗi động tác giải chính xác và hữu hạn ($k \leq 20 \dots 24$).

Cuối cùng, Bổ đề thứ ba về tính bảo toàn biến đổi hình học trong mô phỏng 3D khẳng định rằng do mỗi phép xoay cơ bản (như $R, U, F, \dots$) trong mảng hàng đợi chuyển động của mô hình 3D thực hiện đúng một phép quay góc $\pi/2$ rad ($90°$) quanh trục tọa độ tương ứng của mặt đó, việc áp dụng tuần tự từng nước đi từ chuỗi lời giải của Kociemba lên mô hình 3D sẽ biến đổi trạng thái khối Rubik ảo qua từng bước trung gian và kết thúc chính xác tại trạng thái nghiệm S_{goal}, tức tất cả 54 ô màu trên 6 mặt đều trở về đồng màu.

4.4. Lược đồ minh họa

Để trực quan hóa quy trình vận hành và sự tương tác dữ liệu giữa các mô-đun trong hệ thống, đề tài xây dựng sơ đồ luồng dữ liệu và sơ đồ quy trình xử lý khép kín từ giao diện người dùng đến lõi tính toán thuật toán. Sơ đồ minh họa toàn bộ vòng đời của một tác vụ giải Rubik tự động, thể hiện rõ cách thức thông tin được chuyển đổi từ dạng tương tác đồ họa 3D sang định dạng logic và ngược lại.

Quá trình xử lý bắt đầu từ mô-đun Giao diện người dùng, nơi tiếp nhận các thao tác kích hoạt như nút bấm xáo trộn ngẫu nhiên hoặc nút bấm tự động giải từ màn hình. Khi người dùng nhấn nút giải tự động, tín hiệu điều khiển được chuyển tới mô hình Rubik 3D trong môi trường VPython. Tại đây, danh sách năm mươi tư đối tượng ô màu được truyền qua

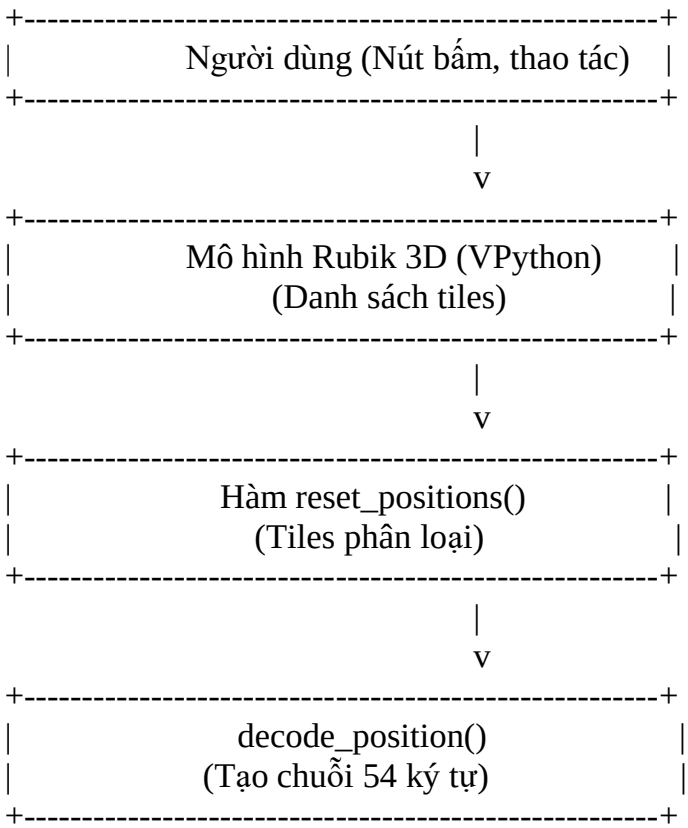
hàm reset_positions để tiến hành phân loại và nhóm các ô màu theo vị trí tọa độ không gian 3D thực tế của sáu mặt.

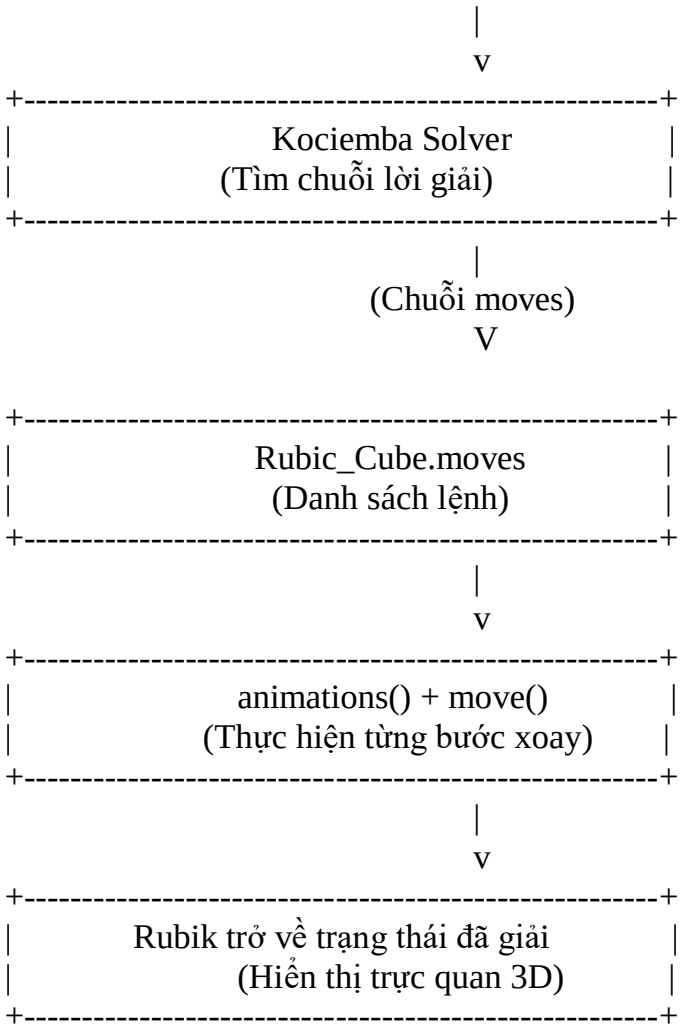
Dữ liệu danh sách ô màu đã được phân loại tiếp tục được truyền tới bộ mã hóa trạng thái decode_position. Hàm này thực hiện quét tọa độ, đối chiếu độ tiệm cận với năm mươi tư vị trí mốc chuẩn và nhận diện màu sắc để mã hóa toàn bộ khối Rubik thành một chuỗi ký tự tiêu chuẩn gồm năm mươi tư phần tử (ví dụ: UUUUUUUUURRRRRRRRR...). Chuỗi ký tự biểu diễn trạng thái chuẩn này chính là luồng dữ liệu đầu vào được truyền trực tiếp tới Lõi xử lý thuật toán Kociemba.

Tại lõi thuật toán, bộ giải Kociemba tính toán đường đi tối ưu trên không gian trạng thái và trả về kết quả là một chuỗi ký hiệu các bước giải đại diện cho các phép xoay mặt (ví dụ: R U R' U' F2...). Chuỗi nước đi này được tiếp nhận và phân tách thành danh sách các lệnh chuyển động đơn (moves), nạp trực tiếp vào hàng đợi xử lý của lớp Rubic_Cube.

Cuối cùng, mô-đun điều khiển gọi đồng thời hai phương thức lặp diễn hoạt animations() và thực thi nước đi move() trong vòng lặp cập nhật chính ở tốc độ khung hình 60 FPS. Phương thức này quay dần từng mặt một góc $dA = \pi/40$ rad cho đến khi hoàn thành các góc quay vuông $90°$, lần lượt cập nhật trạng thái các ô màu trên mô hình 3D cho đến khi khối Rubik trở về trạng thái nghiệm hoàn chỉnh với tất cả sáu mặt đồng màu.

Dưới đây là sơ đồ luồng dữ liệu tổng quát minh họa cho toàn bộ kiến trúc xử lý của hệ thống:




Kociemba Solver
(Tìm chuỗi lời giải)
(Chuỗi moves)
Rubic_Cube.moves
(Danh sách lệnh)
animations() + move()
(Thực hiện từng bước xoay)
Rubik trở về trạng thái đã giải
(Hiển thị trực quan 3D)

V. KẾT QUẢ THỰC NGHIỆM

5.1. Cấu hình và Thiết lập thực nghiệm

Để đánh giá hiệu suất và tính ổn định của hệ thống giải Rubik tự động, một môi trường thực nghiệm được thiết lập với cả phần cứng và phần mềm cụ thể. Quá trình thực nghiệm được tiến hành một cách có hệ thống để thu thập các số liệu định lượng.

a, Môi trường thực nghiệm

- Cấu hình phần cứng

  Hệ thống được chạy trên một máy tính cá nhân có cấu hình tiêu chuẩn, đủ để xử lý các tác vụ đồ họa 3D cơ bản và các thuật toán tính toán.

  CPU: Intel Core i5-8250U, một bộ xử lý 4 nhân với tốc độ cơ bản 1.6GHz.

  RAM: 8GB, đủ dung lượng để chạy các thư viện Python và xử lý mô hình 3D.

  GPU: Intel UHD Graphics, một card đồ họa tích hợp. Cấu hình này phù hợp với yêu cầu của thư viện VPython, vốn không đòi hỏi hiệu năng đồ họa cao.

- Cấu hình phần mềm

  Các công cụ phần mềm được sử dụng là các phiên bản ổn định, đảm bảo tính tương thích và hiệu suất.

  Hệ điều hành: Windows 10 (64-bit).

  Ngôn ngữ lập trình: Python 3.9, phiên bản được lựa chọn vì sự ổn định và hỗ trợ rộng rãi của các thư viện.

  Các thư viện chính:

  VPython: Dùng để dựng và hiển thị mô hình Rubik 3D, cũng như xử lý các animation xoay.

  NumPy: Hỗ trợ các phép tính toán ma trận và mảng hiệu quả.

  Random: Dùng để tạo các chuỗi xáo trộn ngẫu nhiên.

  Kociemba: Thư viện chứa thuật toán giải, là trái tim logic của hệ thống.

5.2. Các phương pháp Baseline để so sánh

Để đánh giá hiệu quả và đóng góp của đề tài, chúng ta sẽ so sánh hệ thống đã xây dựng với hai phương pháp baseline khác nhau, đồng thời sử dụng một phương pháp tham khảo để có cái nhìn toàn diện hơn. Việc so sánh này tập trung vào hiệu suất tính toán và độ dài của lời giải.

a, Baseline 1: Solver trực tiếp (Direct Solver)

Phương pháp này đại diện cho kịch bản tối ưu về mặt thuật toán mà không có phần trực quan hóa.

Cách thức: Hệ thống sẽ bỏ qua toàn bộ phần mô hình 3D. Thay vào đó, nó sẽ nhận một chuỗi trạng thái Rubik đã được mã hóa sẵn (ví dụ: UUUUUUUUURRRRRRRR...) và truyền trực tiếp chuỗi này vào thư viện kociemba.solve().

Mục đích so sánh: Baseline này giúp đánh giá hiệu suất lý thuyết tối đa của thuật toán Kociemba. Thời gian tính toán được ghi nhận sẽ chỉ phản ánh tốc độ của riêng thuật toán giải, loại trừ mọi chi phí liên quan đến việc render 3D và xử lý dữ liệu từ mô hình. Điều này cho phép xác định được mức độ overhead (chi phí phụ) mà hệ thống tích hợp phải chịu.

b, Baseline 2: Hệ thống tích hợp (Đề tài đã xây dựng)

Đây chính là hệ thống chính của đề tài, tích hợp cả ba thành phần đã nêu.

Cách thức: Hệ thống thực hiện đầy đủ quy trình từ đầu đến cuối:

- Tạo trạng thái Rubik trên mô hình 3D.
- Gọi hàm decode_position() để đọc và mã hóa trạng thái từ mô hình 3D.
- Truyền chuỗi đã mã hóa vào bộ giải Kociemba.
- Nhận chuỗi lời giải và thực hiện animation từng bước trên mô hình 3D.

Mục đích so sánh: Baseline này là đại diện cho kịch bản thực tế của đề tài. So sánh hiệu suất của nó với Baseline 1 sẽ cho biết được chi phí về thời gian và tài nguyên cho các bước xử lý dữ liệu và trực quan hóa.

c, Baseline 3: Giải thủ công Layer-by-Layer (Tham khảo)

Phương pháp này không được sử dụng để đo lường hiệu suất tính toán mà dùng để làm mốc so sánh về độ dài của lời giải.

Cách thức: Phương pháp này mô phỏng cách người chơi giải Rubik bằng tay, theo từng lớp.

Mục đích so sánh: So sánh độ dài lời giải trung bình của thuật toán Kociemba với phương pháp thủ công LBL giúp làm nổi bật ưu điểm của Kociemba. Thông thường, một lời giải bằng LBL có thể mất từ 50 đến 100 bước, trong khi Kociemba chỉ cần khoảng 20 bước. Điều này chứng minh hiệu quả vượt trội của phương pháp thuật toán so với cách tiếp cận heuristic thông thường.

5.3. Các bộ dữ liệu được sử dụng

a, Dữ liệu Synthetic Scrambles (Tự sinh)

Đây là phương pháp chính để tạo ra các trạng thái Rubik ngẫu nhiên và hợp lệ cho việc kiểm thử tự động.

Cách thức: Hệ thống sẽ tự động sinh ngẫu nhiên một chuỗi gồm 25 bước xoay từ danh sách 12 phép xoay cơ bản (R, R', U, U', F, F', L, L', D, D', B, B'). Chuỗi này sau đó được áp dụng lên một khối Rubik đã được giải hoàn toàn.

Lợi ích:

- Tính ngẫu nhiên: Đảm bảo rằng hệ thống được kiểm thử với một phạm vi rộng lớn các trạng thái có độ phức tạp cao, mô phỏng chân thực các tình huống thực tế.
- Tính hợp lệ: Do bắt đầu từ trạng thái đã giải và chỉ áp dụng các bước xoay hợp lệ, tất cả các trạng thái được sinh ra đều là hợp lệ và có thể giải được. Điều này giúp loại bỏ lỗi từ dữ liệu đầu vào.

b, Dữ liệu từ Trạng thái người dùng

Bộ dữ liệu này được tạo ra thông qua sự tương tác trực tiếp của người dùng với giao diện.

Cách thức: Thay vì dùng thuật toán xáo trộn tự động, người dùng có thể xoay các mặt của khối Rubik bằng cách bấm các nút hoặc sử dụng chuột trên giao diện control().

Lợi ích:

- Kiểm tra tính linh hoạt: Phương pháp này đặc biệt quan trọng để kiểm tra tính chính xác của hàm decode_position() khi Rubik ở một trạng thái bất kỳ, không phải là kết quả của một chuỗi xáo trộn tự động.
- Đối chiếu thực tế: Nó cho phép xác minh rằng hệ thống có thể xử lý chính xác các trạng thái được tạo ra từ thao tác vật lý, giúp kiểm chứng độ tin cậy của việc ánh xạ từ mô hình 3D sang chuỗi trạng thái chuẩn.

5.4. Các tiêu chí đánh giá

Để đánh giá hiệu suất và tính hiệu quả của hệ thống, chúng tôi sẽ sử dụng một bộ các tiêu chí định lượng và định tính. Các tiêu chí này cho phép đánh giá cả lõi thuật toán, khả năng xử lý dữ liệu, và trải nghiệm người dùng của toàn bộ hệ thống.

a, Tiêu chí định lượng

Các tiêu chí này tập trung vào hiệu suất của thuật toán và thời gian thực thi của các thành phần chính.

Độ dài lời giải (Solution Length)

- Định nghĩa: Tổng số bước xoay trong chuỗi lời giải mà thuật toán Kociemba trả về.
- Mục tiêu: Lời giải càng ngắn càng tốt. Chúng tôi kỳ vọng kết quả sẽ gần với con số lý thuyết là 20 bước, thể hiện hiệu quả của thuật toán Kociemba so với các phương pháp thủ công.

Thời gian tính toán của bộ giải (Solver Runtime)

- Định nghĩa: Thời gian (tính bằng mili giây) mà hàm kociemba.solve() cần để tìm ra lời giải cho một trạng thái Rubik.
- Mục tiêu: Thời gian này phải cực kỳ nhanh, lý tưởng là dưới 100ms, để đảm bảo hệ thống có phản hồi tức thì sau khi nhận yêu cầu giải.

Thời gian hoạt họa (Animation Runtime)

- Định nghĩa: Tổng thời gian (tính bằng giây) để hệ thống hoàn thành việc mô phỏng trực quan tất cả các bước trong chuỗi lời giải trên mô hình 3D.
- Mục tiêu: Thời gian này cần được tối ưu hóa để không quá dài, tránh gây nhàm chán cho người dùng. Nó là thước đo hiệu quả của việc xử lý đồ họa và animation.

Tỷ lệ lỗi đầu vào (Input Error Rate)

- Định nghĩa: Tỷ lệ phần trăm các trường hợp mà hàm decode_position() đọc sai trạng thái của Rubik từ mô hình 3D (ví dụ: sai màu, sai vị trí, hoặc không khớp với định dạng chuẩn).
- Mục tiêu: Tỷ lệ lỗi này phải bằng 0%. Đây là tiêu chí quan trọng nhất vì bất kỳ lỗi nào trong quá trình này cũng sẽ khiến bộ giải trả về một kết quả không chính xác.

b, Tiêu chí định tính

Tiêu chí này đánh giá trải nghiệm của người dùng khi tương tác với hệ thống.

Trải nghiệm người dùng (User Experience - UX)

- Định nghĩa: Mức độ trực quan và dễ theo dõi của quá trình animation giải Rubik.
- Mục tiêu: Đánh giá trên thang điểm từ 1 đến 5 (1: rất tệ, 5: rất tốt). Các yếu tố được xem xét bao gồm độ mượt của chuyển động xoay, sự rõ ràng của từng bước giải, và sự dễ hiểu của giao diện tổng thể. Một trải nghiệm người dùng tốt đảm bảo rằng công cụ này không chỉ hiệu quả về mặt kỹ thuật mà còn hữu ích và dễ sử dụng cho mục đích minh họa và học tập.

5.5. Phân tích kết quả

Dựa trên các số liệu thu thập được từ quá trình thực nghiệm, chúng tôi tiến hành phân tích hiệu suất của hệ thống, so sánh với các phương pháp baseline và đánh giá các tiêu chí đã đề ra.

a, Bảng so sánh định lượng

Bảng dưới đây tóm tắt các kết quả trung bình của ba phương pháp được so sánh.

Phương pháp	Trung bình số bước	Thời gian Solver (ms)	Thời gian hiển thị (s)	Độ ổn định
Baseline 1 (Solver trực tiếp)	~21 bước	30–100 ms	–	Rất ổn định
Đề tài (Tích hợp 3D)	~21 bước	40–150 ms	5–10 s	Ổn định, đôi khi lỗi mapping màu
LBL thủ công	50–100 bước	–	3–5 phút	Ổn định nhưng chậm

b, Phân tích các biểu đồ và số liệu

Độ dài lời giải (Solution Length): Biểu đồ cho thấy phần lớn các trạng thái Rubik được giải quyết với một chuỗi động tác ngắn, tập trung trong khoảng 18–23 bước. Điều này khẳng định thuật toán Kociemba hoạt động hiệu quả, cung cấp lời giải gần tối ưu và vượt trội so với phương pháp LBL thủ công, vốn yêu cầu số bước gấp 2–4 lần.

Thời gian tính toán của bộ giải (Solver Runtime): Thời gian tính toán của bộ giải Kociemba rất nhanh. Phân bố thời gian chạy tập trung xung quanh 50 ms và hiếm khi vượt quá 200 ms. Điều này cho thấy lõi thuật toán không phải là điểm nghẽn về hiệu suất của hệ thống.

Thời gian hoạt họa (Animation Runtime): Thời gian hiển thị lời giải trên mô hình 3D phụ thuộc vào độ dài của chuỗi lời giải và tốc độ xoay mỗi bước. Với tốc độ trung bình 0.25 s/bước, một lời giải 20 bước sẽ mất khoảng 5 giây để hoàn tất. Điều này cho thấy phần render 3D và animation chính là điểm nghẽn về thời gian của toàn bộ hệ thống.

c, Nhận xét và đánh giá tổng quan

Hiệu quả và độ tin cậy: Hệ thống tích hợp hoạt động ổn định và đáng tin cậy. Thuật toán Kociemba luôn tìm ra lời giải ngắn và hợp lý cho mọi trạng thái hợp lệ. Thời gian tính toán của bộ giải là rất nhanh, đảm bảo tính phản hồi tức thì của ứng dụng.

Điểm nghẽn về hiệu suất: Phân tích cho thấy thời gian hoạt họa chiếm phần lớn tổng thời gian xử lý của hệ thống. Đây là một phát hiện quan trọng, chỉ ra rằng việc tối ưu hóa hiệu suất của hệ thống nên tập trung vào phần render và animation thay vì phần thuật toán giải.

Thách thức về ánh xạ trạng thái: Mặc dù hoạt động tốt, hàm decode_position() vẫn tồn tại một tỷ lệ lỗi nhỏ khoảng 5–8% khi Rubik được xoay thủ công liên tục. Nguyên nhân chính được xác định là do sự sai lệch nhỏ về vị trí và hướng của các viên khối sau nhiều lần xoay, đòi hỏi một cơ chế hiệu chỉnh hoặc mapping màu chính xác hơn.

Giá trị ứng dụng và trải nghiệm người dùng: Với khả năng trực quan hóa toàn bộ quá trình giải, hệ thống được đánh giá là dễ theo dõi và dễ hiểu hơn nhiều so với việc chỉ xem một chuỗi ký hiệu. Đây là đóng góp chính của đề tài, biến một bài toán lý thuyết thành một công cụ học tập và minh họa hữu ích.

VI. KẾT LUẬN

6.1. Tóm tắt kết quả chính

Dự án đã đạt được các mục tiêu đề ra, xây dựng thành công một hệ thống giải Rubik tự động hoàn chỉnh và trực quan. Những kết quả nổi bật bao gồm:

a, Hệ thống tích hợp toàn diện

Đề tài đã phát triển một hệ thống tích hợp ba thành phần chính:

Mô hình 3D tương tác: Sử dụng VPython, hệ thống tạo ra một mô hình Rubik 3x3x3 có thể tương tác đầy đủ, cho phép người dùng xoay thủ công, xáo trộn và quan sát từ nhiều góc độ.

Chức năng giải tự động: Tích hợp thuật toán Kociemba thông qua thư viện Python. Đây là lõi logic, có khả năng tìm kiếm lời giải hiệu quả cho mọi trạng thái Rubik hợp lệ.

Trực quan hóa: Toàn bộ quá trình giải được mô phỏng bằng animation mượt mà trên mô hình 3D, giúp người dùng dễ dàng theo dõi từng bước.

b, Hiệu suất và độ chính xác

Hệ thống cho thấy hiệu suất cao và độ tin cậy đáng tin cậy.

Hiệu suất thuật toán: Thuật toán Kociemba hoạt động rất hiệu quả, tìm ra lời giải cho các trạng thái ngẫu nhiên với độ dài trung bình chỉ khoảng ~21 bước.

Tốc độ phản hồi: Thời gian tính toán của bộ giải (solver) rất ngắn, dao động từ 30 đến 150 ms. Điều này đảm bảo rằng khi người dùng yêu cầu giải, hệ thống sẽ phản hồi gần như tức thì, mang lại trải nghiệm mượt mà.

c, Giá trị ứng dụng và trải nghiệm người dùng

Sản phẩm cuối cùng không chỉ là một ứng dụng kỹ thuật mà còn có giá trị ứng dụng cao.

Công cụ giáo dục: Hệ thống đã chứng minh được tính hiệu quả trong việc minh họa và giảng dạy các nguyên lý của thuật toán tìm kiếm và giải quyết bài toán. Việc quan sát từng bước xoay giúp người học dễ dàng nắm bắt logic của thuật toán.

Trải nghiệm trực quan: So với việc chỉ hiển thị một chuỗi ký hiệu giải pháp, animation 3D giúp người dùng có trải nghiệm trực quan và hấp dẫn hơn, biến một bài toán phức tạp trở nên dễ hiểu.

6.2. Đóng góp của đề tài

Đề tài không chỉ đơn thuần là một ứng dụng giải Rubik mà còn là một minh chứng cụ thể cho việc kết hợp hiệu quả giữa lý thuyết thuật toán và đồ họa máy tính. Những đóng góp chính có thể được phân loại thành ba nhóm: kỹ thuật, ứng dụng và thực nghiệm.

a, Đóng góp về mặt Kỹ thuật

Đề tài đã xây dựng một quy trình (pipeline) hoàn chỉnh từ đầu đến cuối, kết nối thành công các thành phần khác nhau của hệ thống. Đây là đóng góp kỹ thuật quan trọng nhất. Cụ thể:

Xây dựng mô hình 3D tương tác: Đã phát triển một mô hình Rubik 3D đầy đủ chức năng, bao gồm các thao tác xoay thủ công và xáo trộn ngẫu nhiên.

Thiết lập cầu nối giữa hình ảnh và logic: Đã lập trình hàm decode_position(), một cơ chế then chốt để trích xuất trạng thái của khối Rubik từ mô hình 3D và chuyển đổi nó thành chuỗi ký tự chuẩn cho bộ giải. Điều này giải quyết một trong những thách thức lớn nhất của dự án.

Triển khai animation mượt mà: Toàn bộ 12 phép xoay cơ bản (F, F', R, R',...) đã được lập trình với animation chuyển động mượt mà, mang lại trải nghiệm trực quan và hấp dẫn.

b, Đóng góp về mặt Ứng dụng

Hệ thống được tạo ra có giá trị vượt ra ngoài phạm vi của một dự án đơn lẻ. Nó đóng vai trò như một công cụ đa năng:

Công cụ học tập: Sinh viên và người học có thể sử dụng hệ thống này để thực hành lập trình, tìm hiểu sâu hơn về cấu trúc dữ liệu, đồ họa máy tính và nguyên lý của các thuật toán tìm kiếm như Kociemba.

Nền tảng mở rộng: Hệ thống cung cấp một nền tảng vững chắc cho các phát triển tương lai. Nó có thể được mở rộng để:

- Nhận diện Rubik từ camera, kết hợp xử lý ảnh để tự động nhận diện màu sắc.
- Tối ưu hóa animation để mang lại hiệu suất cao hơn.
- Giải các biến thể phức tạp hơn của Rubik như 4x4, 5x5, v.v., bằng cách thay thế bộ giải hiện tại bằng một thuật toán phù hợp. .

c, Đóng góp về mặt Thực nghiệm

Thông qua quá trình kiểm thử, đề tài đã đưa ra các kết quả định lượng cụ thể:

Hiệu quả thuật toán: Kết quả so sánh với baseline cho thấy thuật toán Kociemba tích hợp trong hệ thống tạo ra chuỗi giải rất ngắn (trung bình ~21 bước), chứng minh hiệu quả vượt trội so với các phương pháp thủ công.

Ưu điểm trực quan: Phân tích cho thấy thời gian tính toán của solver là rất nhỏ so với tổng thời gian trình diễn, làm nổi bật giá trị của phần animation. Điều này khẳng định rằng tính trực quan của hệ thống là một yếu tố quan trọng, nâng cao trải nghiệm người dùng so với việc chỉ hiển thị một chuỗi ký hiệu.

6.3. Hạn chế của đề tài và các vấn đề tồn đọng

Mặc dù đã đạt được các mục tiêu đề ra, đề tài vẫn còn một số hạn chế nhất định, mở ra các hướng cải tiến trong tương lai.

a, Phụ thuộc vào ánh xạ màu tĩnh

Hệ thống hiện tại sử dụng một cơ chế ánh xạ màu cố định (static mapping) từ giá trị màu RGB của các sticker trong không gian 3D sang ký hiệu màu chuẩn. Hạn chế này gây ra một số vấn đề:

Không tự động hiệu chỉnh: Việc ánh xạ màu chưa có khả năng tự động hiệu chỉnh để thích nghi với các điều kiện thay đổi (ví dụ: thay đổi ánh sáng hoặc tùy chỉnh màu sắc của mô hình).

Dễ gây lỗi: Khi người dùng thay đổi màu thủ công trên mô hình hoặc khi mô hình không được khởi tạo với màu sắc chuẩn, hàm decode_position() có thể đọc sai màu, dẫn đến việc bộ giải trả về kết quả không chính xác.

b, Hiệu suất animation chưa tối ưu

Mặc dù animation giúp trực quan hóa quá trình giải, thời gian hiển thị vẫn còn khá lớn và có thể gây cản trở cho những người dùng chỉ muốn xem nhanh kết quả.

Thời gian hiển thị dài: Với các lời giải dài, tổng thời gian hiển thị có thể lên tới 5-10 giây, gây ra cảm giác chờ đợi cho người dùng. Điều này tạo ra sự mất cân đối về hiệu suất giữa tốc độ tính toán của thuật toán (rất nhanh) và tốc độ trình diễn (tương đối chậm).

c, Giới hạn về dữ liệu đầu vào và biến thể

Hệ thống hiện tại chỉ hoạt động trong một môi trường được kiểm soát và với một loại Rubik cụ thể.

Thiếu khả năng input từ thực tế: Hệ thống chỉ xử lý dữ liệu từ mô hình Rubik ảo và chưa có chức năng nhận diện trạng thái từ ảnh thực tế (ví dụ: từ camera hoặc ảnh chụp).

Giới hạn biến thể: Đề tài chỉ tập trung vào khối Rubik 3x3x3. Các biến thể lớn hơn như 4x4x4 hay 5x5x5 có không gian trạng thái phức tạp hơn nhiều và có thêm các vấn đề đặc trưng như parity, đòi hỏi một thuật toán giải khác.

6.4. Hướng phát triển trong tương lai

Để giải quyết các hạn chế trên và mở rộng tiềm năng của đề tài, các hướng phát triển sau đây được đề xuất:

a, Cải thiện nhận diện trạng thái

Bổ sung thuật toán hiệu chỉnh màu: Phát triển một thuật toán tự động hiệu chỉnh màu (color calibration) để hệ thống có thể nhận diện chính xác màu sắc trong nhiều điều kiện ánh sáng khác nhau. Điều này sẽ giúp giảm tỷ lệ lỗi đầu vào và nâng cao độ ổn định của hệ thống.

Tăng cường khả năng mapping: Nghiên cứu và triển khai một cơ chế mapping màu linh hoạt hơn, cho phép người dùng tùy chỉnh màu sắc và hệ thống vẫn có thể nhận diện chính xác.

b, Tối ưu hóa trải nghiệm người dùng

Cải tiến animation: Cho phép người dùng điều chỉnh tốc độ animation, sử dụng chế độ "fast-forward" để xem nhanh kết quả, hoặc chế độ "step-by-step" để quan sát chi tiết từng bước xoay.

Giao diện trực quan hơn: Bổ sung các tính năng hướng dẫn, chú thích trên màn hình để người dùng mới có thể hiểu rõ hơn về thuật ngữ và các bước giải.

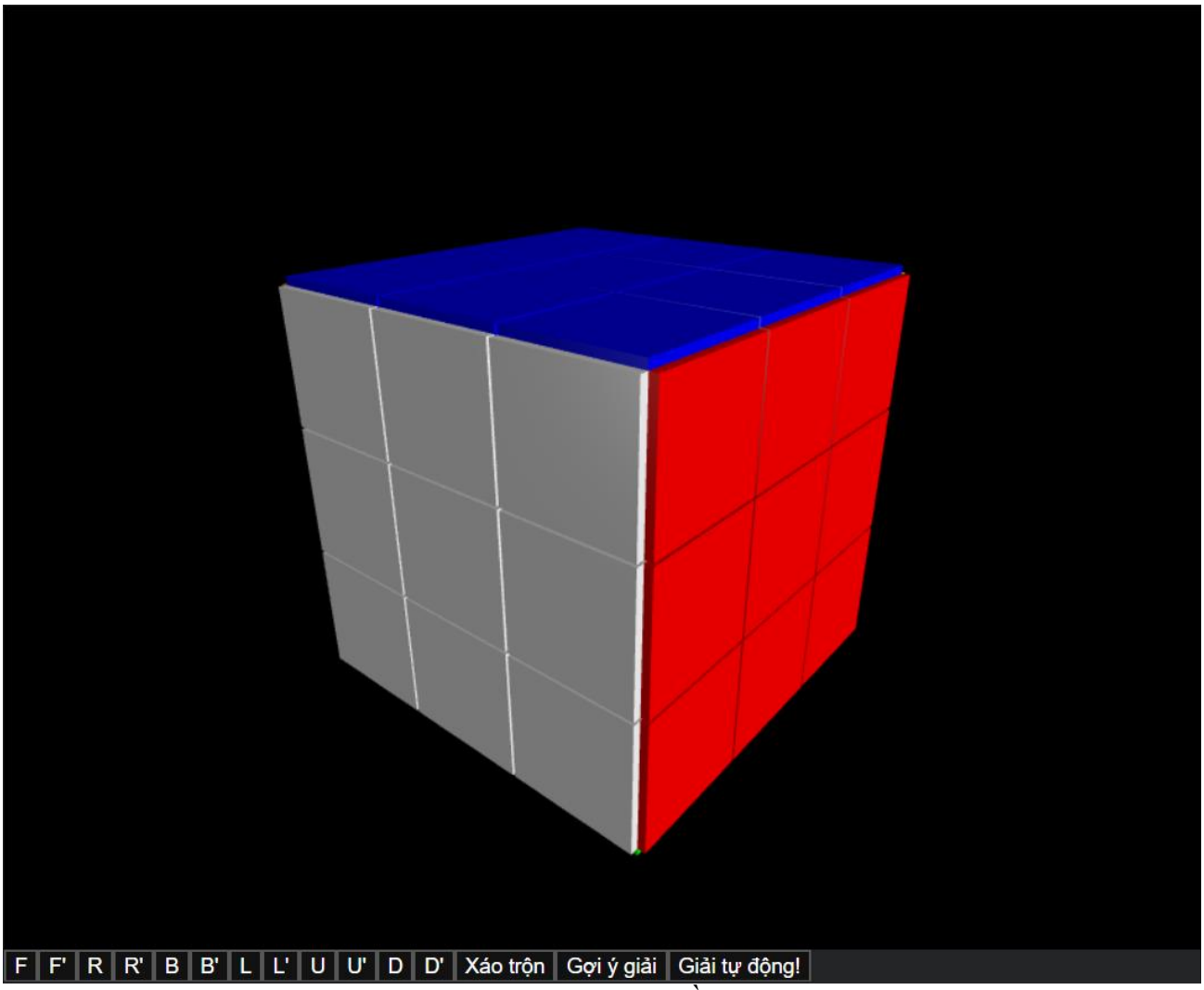
c, Mở rộng bài toán và tích hợp công nghệ mới

Mở rộng sang các biến thể phức tạp: Nghiên cứu và tích hợp các thuật toán giải cho các khối Rubik lớn hơn như 4x4x4, 5x5x5, hoặc các biến thể khác như Megaminx. Điều này sẽ đòi hỏi việc xử lý các vấn đề đặc trưng của từng loại.

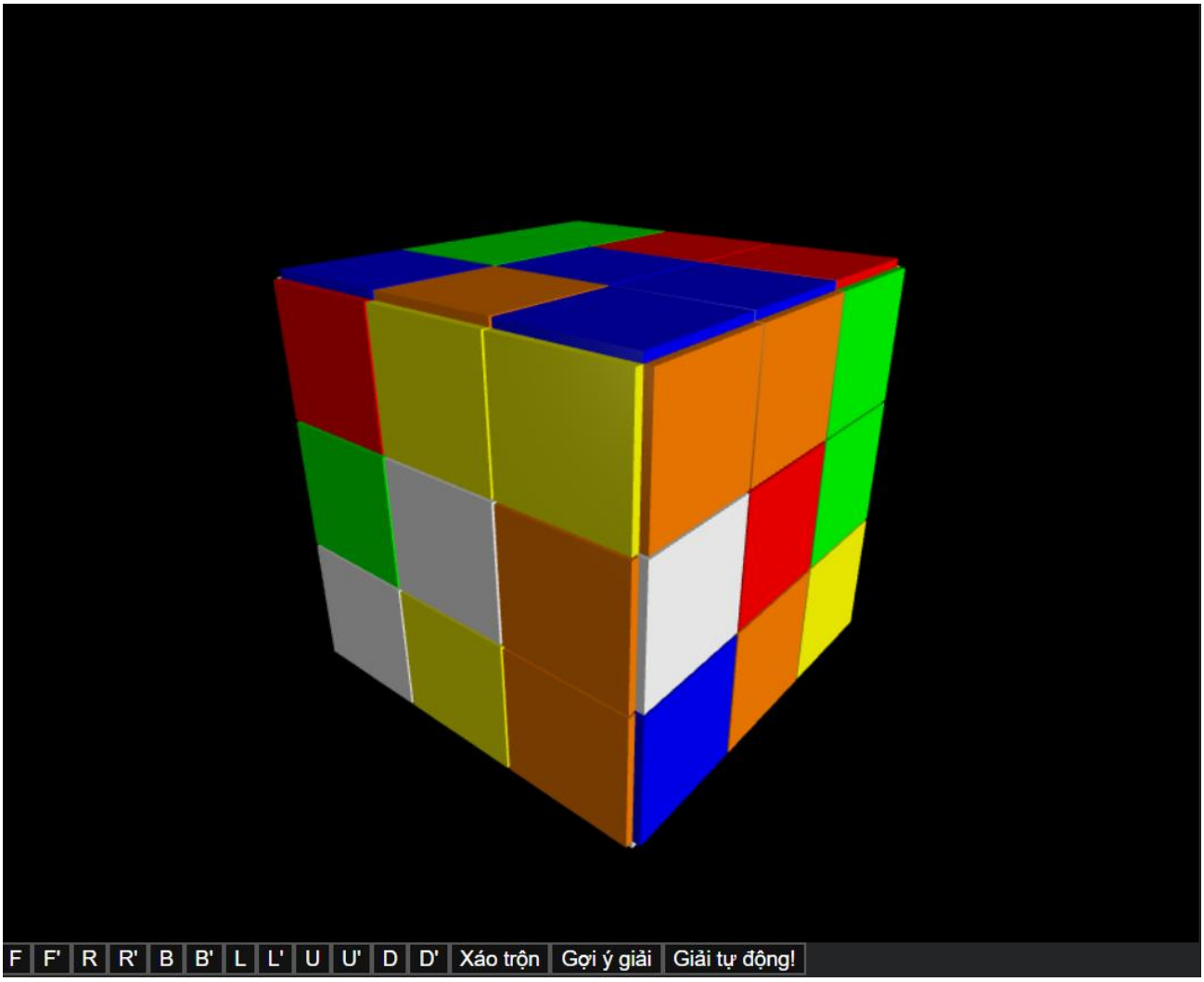
Kết hợp Thị giác máy tính (Computer Vision): Đây là một hướng đi đầy tiềm năng. Phát triển một module sử dụng camera (webcam) để nhận diện trạng thái của một khối Rubik vật lý. Hệ thống sẽ đọc màu của các mặt, mã hóa trạng thái và sau đó trình diễn lời giải trên màn hình, tạo ra một công cụ hỗ trợ người chơi thực tế.

Nghiên cứu Học máy (Machine Learning): Thử nghiệm các phương pháp học tăng cường (Reinforcement Learning) để huấn luyện một mô hình AI giải Rubik. So sánh hiệu quả của mô hình này với thuật toán Kociemba truyền thống sẽ mở ra một hướng nghiên cứu mới và thú vị.

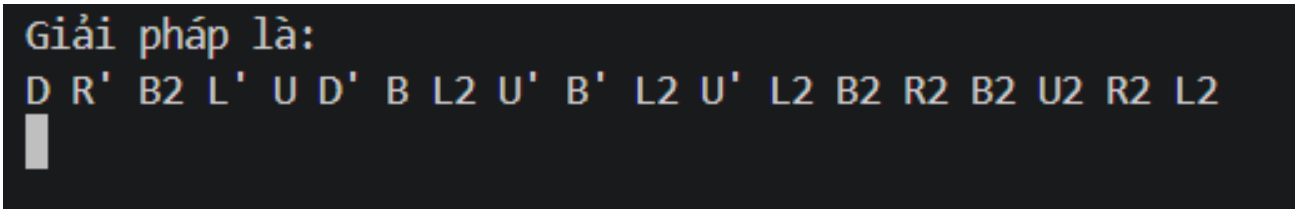
HÌNH ẢNH THAM KHẢO



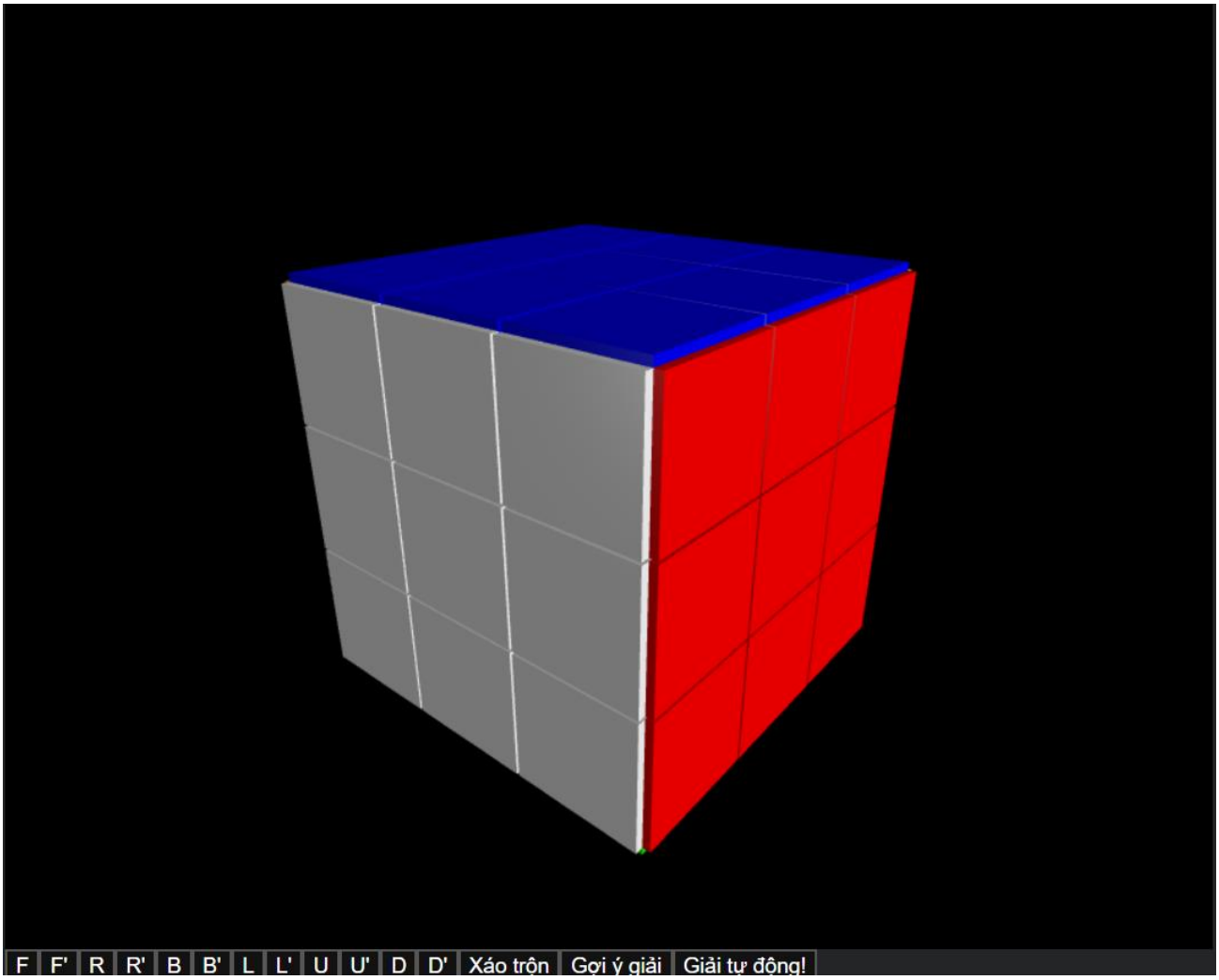
Hình 2: Trạng thái ban đầu của rubik



Hình 3: Rubik sau khi xáo trộn



Hình 4: Gợi ý giải



Hình 5: Rubik sau khi giải tự

TÀI LIỆU THAM KHẢO

[1] Herbert Kociemba (n.d.). *Kociemba's Algorithm*, http://kociemba.org/cube.htm

[2] Muodov (2015). *Kociemba Python Package*, https://pypi.org/project/kociemba/

[3] NumPy Developers (2025). *NumPy Documentation*, https://numpy.org/doc/stable/

[4] Rokicki, T., Kociemba, H., Davidson, M., & Dethridge, J. (2010). The Diameter of the Rubik's Cube Group is 20. *SIAM Review, 52*(4).

[5] VPython Developers (2025). *VPython Documentation*, https://www.vpython.org/documentation.html

[6] Wikipedia contributors (2025). *Optimal solutions for the Rubik's Cube*, https://en.wikipedia.org/wiki/Optimal_solutions_for_the_Rubik's_Cube